

Jenkins CI/CD Lab — Practice Questions (Set 2)

ISWE406L — Agile Development Process and DevOps | Windows Agent

Question 1

Create a Jenkins pipeline on a Windows agent with three stages: **Checkout** from your GitHub repository, **Install Dependencies** from a `requirements.txt` file, and **Run Unit Tests** with `pytest`. Write an `app.py` containing `square(n)` and `is_even(n)`, and a `test_app.py` with two test functions for each. Run the build and confirm from the Console Output how many tests passed. Then change `is_even` so it returns the wrong result, push, rebuild, and show the stage at which the pipeline stops.

Question 2

Build a pipeline with the stages **Checkout**, **Install Dependencies**, and **Run Unit Tests (verbose)**. Write an `app.py` with `sum_list(numbers)` and `count_negatives(numbers)`. In `test_app.py`, use `@pytest.mark.parametrize` to test `sum_list` against four input cases and `count_negatives` against three, and run `pytest` with the `-v` flag. State how many tests the Console Output reports and how each parametrized case is labelled. Then add a case with a wrong expected value to `count_negatives` and show which tests pass and which fail.

Question 3

Design a pipeline that uses a choice parameter named `ENVIRONMENT` with the options `dev`, `staging`, and `prod`. The stages should be **Checkout**, **Build** (a compile check of `app.py` using `py_compile`), and **Deploy**. The Deploy stage must pause with an input step that asks "Approve deployment to <ENVIRONMENT>?" and has an OK button labelled "Go", and then run `app.py`. Run one build choosing `staging` and clicking **Go**, and another choosing `prod` and clicking **Abort**. Explain the difference between choosing a parameter before the run and approving an input during the run.

Question 4

Create a pipeline with the stages **Checkout**, **Generate Report**, and **Archive Report**. Before generating the report, print the built-in variables `BUILD_NUMBER`, `JOB_NAME`, and `WORKSPACE`. The `app.py` script should write a `build_report.txt` file listing a course name and the number of students enrolled, and the file must be archived with `archiveArtifacts` and `fingerprint: true`. Run two builds, changing the student count in between, and show that each build keeps its own copy of the report.

Question 5

Write a pipeline with the stages **Checkout**, **Parallel Checks**, and **Summary**. Inside **Parallel Checks**, run `unit_check.py` and `integration_check.py` side by side, where each script prints a start message, sleeps for 5 seconds, and prints a pass message. Add a post block with separate success and failure messages. Estimate how long the parallel stage takes compared with running the scripts one after another. Then introduce a syntax error into `integration_check.py`, rebuild, and explain whether the **Summary** stage runs and which post message appears.

Question 6

Design a pipeline with a custom environment block defining `APP_NAME` and `APP_VERSION`, and a `booleanParam` named `SEND_EMAIL`. The stages should be **Checkout**, **Build** (which compiles `app.py` and echoes the app name and version), and **Send Notification**. The Send Notification stage runs only when `SEND_EMAIL` is true, which you should control with a `when` directive, and uses the `mail` step (or the `echo` workaround) with `APP_NAME` and `APP_VERSION` in the subject. Run builds with the checkbox checked and unchecked, and explain how a skipped stage differs from a failed stage.

Question 7

Build a pipeline with the stages **Checkout**, **Build**, and **Deploy**. The Build stage compiles `app.py`, waits 20 seconds, and then passes `milestone(1)`. The Deploy stage passes `milestone(2)` at its very start and then echoes a deployment message. Click **Build Now** twice in quick succession. Identify which build gets aborted and at which milestone, and explain why. Then explain the extra protection `milestone(2)` gives at the entrance to Deploy, and what could go wrong if both overlapping builds were allowed to deploy without any milestones.